

Appendix

MLOps Principles

Building robust and scalable ML systems requires more than creating powerful models. It demands an all-encompassing approach to operationalizing the entire ML lifecycle. Let's explore the **six core principles** that guide the MLOps field. These principles are independent of any tool and are at the core of building robust and scalable ML systems. They provide a guideline for designing production-ready applications, ensuring consistency, reliability, and scalability at every stage.

With that in mind, let's begin with the foundation: automation or operationalization.

1. Automation or operationalization

To adopt MLOps, there are three core tiers that most applications build up gradually, from manual processing to full automation:

- **Manual process:** The process is experimental and iterative in the early stages of developing an ML application. The data scientist manually performs each pipeline step, such as data preparation and validation, model training, and testing. At this point, they commonly use Jupyter notebooks to train their models. This stage's output is the code used to prepare the data and train the models.
- **Continuous training (CT):** The next level involves automating model training. This is known as continuous training, which triggers model retraining whenever required. At this point, you often automate your data and model validation steps. This step is usually done by an orchestration tool, such as ZenML, that glues all your code together and runs it on specific triggers. The most common triggers are on a schedule, for example, every day or when a specific event comes in, such as when new data is uploaded or the monitoring system detects a drop in performance, offering you the flexibility to adapt to various triggers.
- **CI/CD:** In the final stage, you implement your CI/CD pipelines to enable fast and reliable deployment of your ML code into production. The key advancement at this stage is the automatic building, testing, and deployment of data, ML models, and training pipeline components. CI/CD is used to quickly push new code into various environments, such as staging or production, ensuring efficient and reliable deployment.

As we build our LLM system using the **FTI (feature, training, inference)** architecture, we can quickly move from a manual process to CI/CD/CT. In

Figure A.1, we can observe that the CT process can be triggered by various events, such as a drop in performance detected by the monitoring pipeline or a batch of fresh data arriving. Also, Figure A.1 is split into two main sections; the first one highlights the automated processes, while at the bottom, we can observe the manual processes performed by the data science team while experimenting with various data processing methods and models. Once they improve the model by tinkering with how the data is processed or the model architecture, they push the code to the code repository, which triggers the CI/CD pipeline to build, test, package, and deploy the new changes to the FTI pipelines.

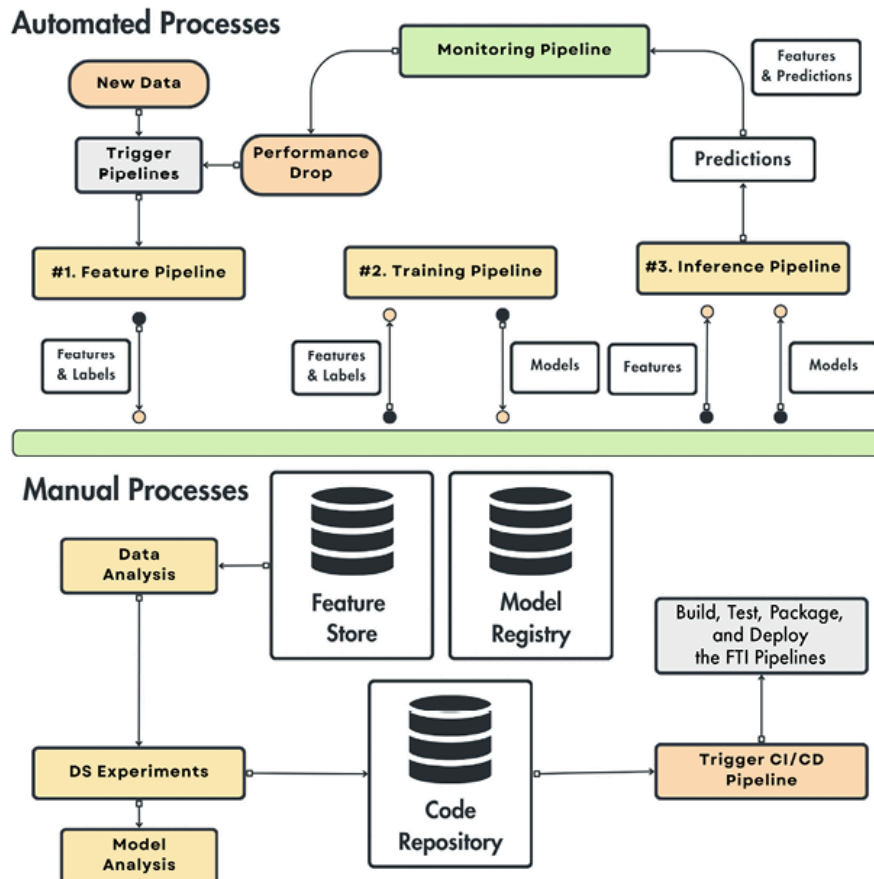


Figure A.1: CI/CD/CT on top of the FTI architecture

To conclude, CT automates the FTI pipelines, while CI/CD builds, tests, and pushes new versions of the FTI pipeline code to production.

2. Versioning

By now, we understand that the whole ML system changes if the code, model, or data changes. Thus, it is critical to track and version these three elements individually. But what strategies can we adopt to track the code, model, and data separately?

- The **code** is tracked by Git, which helps us create a new commit (a snapshot of the code) on every change added to the codebase. Also,

Git-based tools usually allow us to make releases, which typically pack multiple features and bug fixes. While the commits contain unique identifiers that are not human-interpretable, a release follows more common conventions based on their major, minor, and patch versions. For example, in a release with version “v1.2.3,” 1 is the major version, 2 is the minor version, and 3 is the patch version. Popular tools are GitHub and GitLab.

- To version the **model**, you leverage the model registry to store, share, and version all the models used within your system. It usually follows the same versioning conventions used in code releases, defined as **Semantic Versioning**, which, along with the major, minor, and patch versions, also supports alpha and beta releases that signal applications. At this point, you can also leverage the ML metadata store to attach information to the stored model, such as what data it was trained on, its architecture, performance, latency, and whatever else makes sense to your specific use case. Doing so creates a clear catalog of models that can easily be navigated across your team and company.
- Versioning the **data** isn't as straightforward as versioning the code and model because it depends on the type of data you have (structured or unstructured) and the scale of data you have (big or small). For example, for structured data, you can leverage a SQL database with a version column that helps you track the changes in the dataset. However, other popular solutions are based on Git-like systems, such as **Data Version Control (DVC)**, that track every change made to the dataset. Other trendy solutions are based on artifacts similar to a model registry that allows you to add a virtual layer to your dataset, tracking and creating a new version for every change made to your data. Comet.ml, **W&B (Weights & Biases)**, and ZenML offer powerful artifact features. For all solutions, you must store the data on-premises or use cloud object storage solutions such as AWS S3. These tools provide features that allow you to structure your datasets and versions, track, and access them.

3. Experiment tracking

Training ML models is an entirely iterative and experimental process. Unlike traditional software development, it involves running multiple parallel experiments, comparing them based on a set of predefined metrics, and deciding which one should advance to production. An experiment tracking tool allows you to log all the necessary information, such as metrics and visual representations of your model predictions, to compare all your experiments and easily select the best model. Popular tools are Comet ML, W&B, MLflow, and Neptune.

4. Testing

The same trend is followed when testing ML systems. Hence, we must test our application across all three dimensions: the data, the model, and the

code. We must also ensure that the feature, training, and inference pipeline are well integrated with external services, such as the feature store, and work together as a system. When working with Python, the most common tool to write your tests is `pytest`, which we also recommend.

Test types

In the development cycle, six primary types of tests are commonly employed at various stages:

- **Unit tests:** These tests focus on individual components with a single responsibility, such as a function that adds two tensors or one that finds an element in a list.
- **Integration tests:** These tests evaluate the interaction between integrated components or units within a system, such as the data evaluation pipeline or the feature engineering pipeline, and how they are integrated with the data warehouse and feature store.
- **System tests:** System tests play a crucial role in the development cycle as they examine the entire system, including the complete and integrated application. These tests rigorously evaluate the end-to-end functionality of the system, including performance, security, and overall user experience—for example, testing an entire ML pipeline, from data ingestion to model training and inference, ensuring the system produces the correct outputs for given inputs.
- **Acceptance tests:** These tests, often called **user acceptance testing** (UAT), are designed to confirm that the system meets specified requirements, ensuring it is ready for deployment.
- **Regression tests:** These tests check for previously identified errors to ensure that new changes do not reintroduce them.
- **Stress tests:** These tests evaluate the system's performance and stability under extreme conditions, such as high load or limited resources. They aim to identify breaking points and ensure the system can handle unexpected spikes in demand or adverse situations without failing.

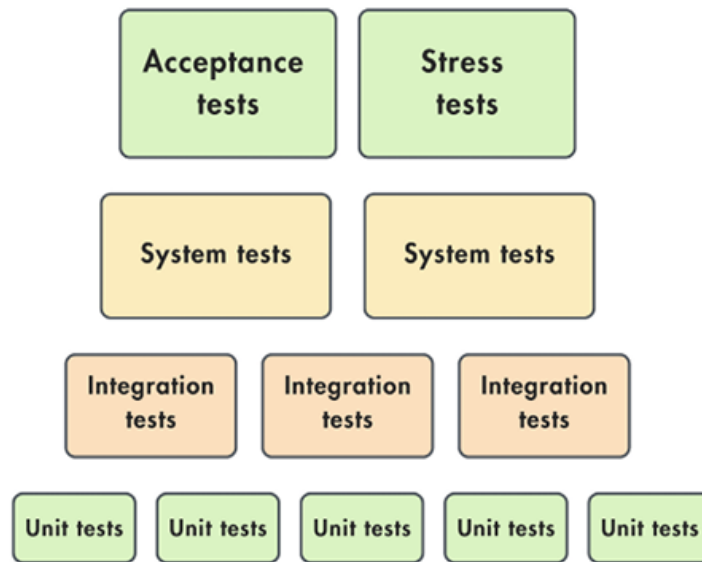


Figure A.2: Test types

We've intentionally left regression tests out of the preceding figure because they aren't a distinct testing phase. Instead, regression testing is applied across all levels—unit, integration, system, acceptance, and stress tests—to ensure that changes don't reintroduce previous errors. It's an ongoing process within these phases, not a separate type of test, which is why it's not shown as a separate category.

What do we test?

When writing most tests, you take a component and treat it as a black box. Thus, what you have control over is the input and output. You want to test that you get an expected output for a given input. With that in mind, here are a few things you should usually test:

- **Inputs:** Data types, format, length, and edge cases (min/max, small/large, etc.)
- **Outputs:** Data types, formats, exceptions, and intermediary and final outputs

Test examples

When testing your code, you can leverage the standards from classic software engineering. Here are a few examples of code tests you can include when writing unit tests to get a better idea of what we want to test at this point—for instance, you want to check that a sentence is cleaned as expected.

Also, you can look at your chunking algorithm and assert that it works properly by using various sentences and chunk sizes.

When we talk about **data tests**, we mainly refer to data validity. Your data validity code usually runs when raw data is ingested from the data warehouse or after computing the features. It is part of the feature pipeline. Thus, by writing integration or system tests for your feature pipeline, you can check that your system responds properly to valid and invalid data.

Testing data validity depends a lot on your application and data type. For example, when working with tabular data, you can check for non-null values, that a categorical variable contains only the expected values, or that a float value is always positive. You can check for length, character encoding, language, special characters, and grammar errors when working with unstructured data such as text.

Model tests are the trickiest, as model training is the most non-deterministic process of an ML system. However, unlike traditional software, ML systems can successfully complete without throwing any errors. However, the real issue is that they produce incorrect results that can only be observed during evaluations or tests. Some standard model test techniques involve checking:

- The shapes of the input and model output tensors
- That the loss decreases after one batch (or more) of training
- Overfit on a small batch, and the loss approaches 0
- That your training pipeline works on all the supported devices, such as the CPU and GPU
- That your early stopping and checkpoint logic works

All the tests are triggered inside the CI pipeline. If some tests are more costly, for example, the model ones, you can execute them only on special terms, such as only when modifying the model code.

At the other end of the spectrum, you can also perform **behavioral testing** on your **model**, which tries to adopt the strategy from code testing and treats the model as a black box while looking solely at the input data and expected outputs. This makes the behavioral testing methods model agnostic. A fundamental paper in this area is *Beyond Accuracy: Behavioral Testing of NLP Models with CheckList*, which we recommend if you want to dig more into the subject. However, as a quick overview, the paper proposes that you test your model against three types of tests. We use a model that extracts the main subject from a sentence as an example:

- **Invariance:** Changes in your input should not affect the output—for example, below is an example based on synonym injection:

```
model(text="The advancements in AI are changing the world rapidly.")  
# output: ai
```

```
model(text="The progress in AI is changing the world rapidly.")  
# output: ai
```

- **Directional:** Changes in your input should affect the outputs—for example, below is an example where we know the outputs should change based on the provided inputs:

```
model(text="Deep learning used for sentiment analysis.")  
# output: deep-learning  
model(text="Deep learning used for object detection.")  
# output: deep-learning  
model(text="RNNs for sentiment analysis.")  
# output: rnn
```

- **Minimum functionality:** The most simple combination of inputs and expected outputs—for example, below is a set of simple examples that we expect the model should always get right:

```
model(text="NLP is the next big wave in machine learning.")  
# output: nlp  
model(text="MLOps is the next big wave in machine learning.")  
# output: mlops  
model(text="This is about graph neural networks.")  
# output: gnn
```



For more on testing, we recommend reading *Testing Machine Learning Systems: Code, Data, and Models* by Goku Mohandas:
<https://madewithml.com/courses/mlops/testing/>.

5. Monitoring

Monitoring is vital for any ML system that reaches production.

Traditional software systems are rule-based and deterministic. Thus, once it is built, it will always work as defined. Unfortunately, that is not the case with ML systems. When implementing ML models, we haven't explicitly described how they should work. We have used data to compile a probabilistic solution, which means that our ML model will constantly be exposed to a level of degradation. This happens because the data from production might differ from the data the model was trained on. Thus, it is natural that the shipped model doesn't know how to handle these scenarios.

We shouldn't try to avoid these situations but create a strategy to catch and fix these errors in time. Intuitively, monitoring detects the model's performance degradation, which triggers an alarm that signals that the

model should be retrained manually, automatically, or with a combination of both.

Why retrain the model? As the model performance degrades due to a drift in the training dataset and what it inputs from production, the only solution is to adapt or retrain the model on a new dataset that captures all the new scenarios from production.

As training is a costly operation, there are some tricks that you can perform to avoid retraining, but before describing them, let's quickly understand what we can monitor to understand our ML system's health.

Logs

The approach to logging is straightforward, which is to capture everything, such as:

- Document the system configurations.
- Record the query, the results, and any intermediate outputs.
- Log when a component begins, ends, crashes, and so on.
- Ensure that each log entry is tagged and identified in a way that clarifies its origin within the system.

While capturing all activities can rapidly increase the volume of logs, you can take advantage of numerous tools for automated log analysis and anomaly detection that leverage AI to efficiently scan all the logs, providing you with the confidence to manage the logs effectively.

Metrics

To quantify your application's healthiness, you must define a set of metrics. Each metric measures different aspects of your application, such as the infrastructure, data, and model.

System metrics

The system metrics are based on monitoring service-level metrics (latency, throughput, error rates) and infrastructure health (CPU/GPU, memory). These metrics are used both in traditional software and ML as they are crucial to understanding whether the infrastructure works well and the system works as expected to provide a good user experience to the end users.

Model metrics

Merely monitoring the system's health won't suffice to identify the deeper issues within our model. Therefore, moving on to the next layer of metrics that focus on the model's performance is crucial. This includes quan-

titative evaluation metrics like accuracy, precision, and F1 score, as well as essential business metrics influenced by the model, such as ROI and click rate.

Analyzing cumulative performance metrics over the entire deployment period is often ineffective. Instead, evaluating performance over time intervals relevant to our application, such as hourly, is essential. Thus, in practice, you window your inputs and compute and aggregate the metrics at the window level. These sliding metrics can provide a clearer picture of the system’s health, allowing us to detect issues more promptly without them being obscured by historical data.

We may not always have access to ground-truth outcomes to evaluate the model’s performance on production data. This is particularly challenging when there is a significant delay or when real-life data requires annotation. To address this issue, we can develop an approximate signal to estimate the model’s performance or label a small portion of our live dataset to assess performance. When talking about ML monitoring, an approximate signal is also known as a **proxy metric**, usually implemented by drift detection methods, which are discussed in the following section.

Drifts

Drifts are proxy metrics that help us detect potential issues with the production model in time without requiring any ground truths/labels. *Table A.1* shows three kinds of drifts.

What drifts	Description	Drift formulation
X	Inputs (features)	data drift $\rightarrow P(X) \neq P_{\text{ref}}(X)$
y	Outputs (ground truths/labels)	target drift $\rightarrow P(y) \neq P_{\text{ref}}(y)$
$P(y X)$	relationship between X and y	concept drift $\rightarrow P(y X) \neq P_{\text{ref}}(y X)$

Table A.1: Relationship between data, model, and code changes

Data drift

Data drift, also called feature drift or covariate shift, occurs when the distribution of the production data deviates from that of the training data, as

shown in *Figure A.3*. This difference means the model cannot handle the changes in feature space, leading to potentially unreliable predictions. Drift can result from natural real-life changes or systemic problems like missing data, pipeline errors, and schema modifications.

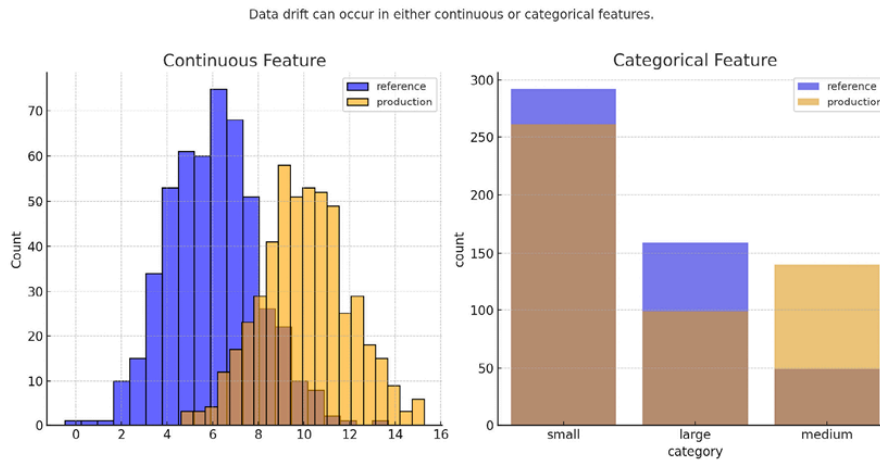


Figure A.3: Data drift examples

When data begins to drift, the degradation in our model's performance might not be immediately noticeable, particularly if the model interpolates effectively. Nevertheless, this presents an ideal chance to consider retraining before the drift affects the model's performance.

Target drift

In addition to changes in input data (data drift), we might also encounter shifts in output distribution. The shift could involve changes in the shape of the distribution or the addition and removal of classes in categorical tasks. While retraining the model can help reduce performance degradation due to target drift, you can often prevent it by adapting the head processing steps and model head to support the new schema of the output class.

For example, if you have a classifier that predicts if an image contains animals or people, and you get a picture with buildings, you can either adapt your model to support an unknown class or adjust the head of the model to add the new class for future predictions.

Concept drift

In addition to changes in input and output data, their relationship can also shift. This phenomenon, known as **concept drift**, makes our model ineffective because the patterns it previously learned to associate inputs with outputs become outdated. As illustrated in the following figure, concept drifts can manifest in various ways:

- Gradually over time

- Suddenly, due to an external event
- Periodically, due to recurring events

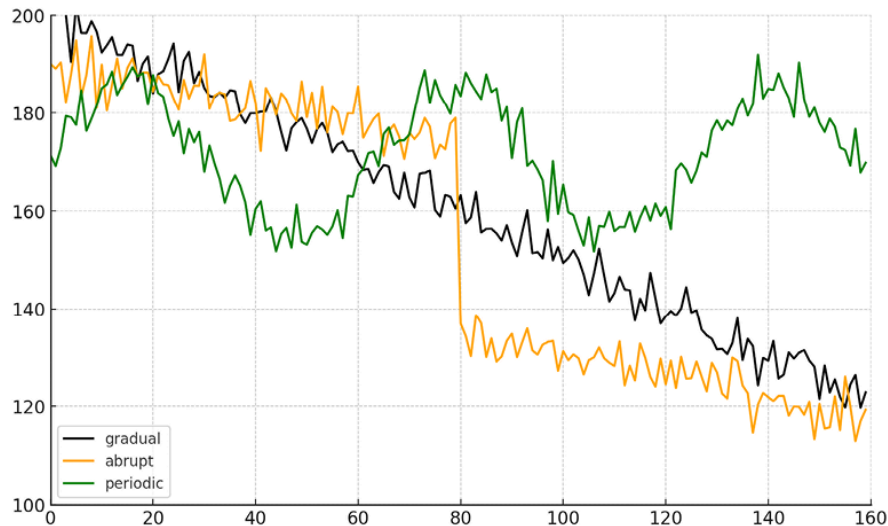


Figure A.4: Concept drift examples

For example, this happens when using the model in a different geographic area. Let's assume you want to build a model that predicts whether a person will buy a specific car. You initially built it for the American market. Now, you want to use it in the European market, where people tend to buy smaller cars, creating a drift between the size feature of the car and the output probability of purchasing the vehicle. Of course, concept drifts can be more subtle than this example.



All these types of drift can happen simultaneously, complicating pinpointing the exact sources of drift.

How to detect and measure drifts

Now that we've recognized the various types of drift, it's crucial to understand how to detect and measure it. To do so, you need two types of windows:

- **A reference window:** This is the collection of data points used as a baseline to compare against the production data distributions for drift identification. It is usually gathered from the training dataset.
- **A test window:** This collects data points gathered while the ML system is in production. It is compared with the reference window to ascertain if drift has occurred.

To measure the drifts, you leverage hypothesis tests that verify the change in distribution between the two windows. For example, you can use the **Kolmogorov-Smirnov (KS)** test to monitor a single continuous feature. This is known as a **univariate (1D)** test. Thus, you must run it for

every feature you want to monitor. You can leverage a chi-squared univariate test to monitor categorical variables and determine if the frequency of events in production is consistent with the reference window distribution.

```
from alibi_detect.cd import KSDrift
cd = KSDrift(X_ref, p_val=.05, preprocess_fn=preprocess_fn, input_shape=(1
```

When working with text data in an embedding representation, we have to model a multivariate distribution, which is how LLMs work with text. A popular approach is to take the embeddings of the test and reference windows, apply a dimensionality reduction algorithm, and apply an algorithm such as **maximum mean discrepancy (MMD)**. This algorithm is a kernel-based approach that measures the distance between two distributions by computing the distance between the mean of the embeddings of the two windows.

```
from alibi_detect.cd import MMDDrift
cd = MMDDrift(x_ref, backend='pytorch', p_val=.05)
preds = cd.predict(x)
```

Monitoring vs. observability

Monitoring involves the collection and visualization of data, whereas observability provides insights into system health by examining its inputs and outputs. For instance, monitoring allows us to track a specific metric to detect potential issues.

On the other hand, a system is considered observable if it generates meaningful data about its internal state, which is essential for diagnosing root causes.

Alerts

Once we define our monitoring metrics, we need a way to get notified. The most common approaches are to send an alarm in the following scenarios:

- A metric passes the values of a static threshold—for example, when the accuracy of the classifier is lower than 0.8, send an alarm.
- Tweaking the p-value of the statistical tests that check for drifts. A lower p-value means a higher confidence that the production distribution differs from the reference one.

These thresholds and p-values depend on your application. However, it is essential to find the correct values, as you don't want to overcrowd your alarming system with false positives. In that case, your alarm system

won't be trustworthy, and you will either overreact or not react at all to issues in your system. Some common channels for sending alarms to your stakeholders are Slack, Discord, your email, and PagerDuty. The system's stakeholders can be the core engineers, managers, or anyone interested in the system.

Depending on the nature of the alarm, you have to react differently. But before taking any action, you should be able to inspect it and understand what caused it. You should inspect what metric triggered the alarm, with what value, the time it happened, and anything else that makes sense to your application.

When the model's performance degrades, the first impulse is to retrain it. But that is a costly operation. Thus, you first have to check that the data is valid, the schema hasn't changed, and the data point was not an isolated outlier. If neither is true, you should trigger the training pipeline and train the model on the newly shifted dataset to solve the drift.

6. Reproducibility

Reproducibility means that every process within your ML systems should produce identical results given the same input. This has two main aspects.

The first one is that you should always know what the inputs are—for example, when training a model, you can use a plethora of hyperparameters. Thus, you need a way to always track what assets were used to generate the new assets, such as what dataset version and config were used to train the model.

The second aspect is based on the non-deterministic nature of ML processes. For example, when training a model from scratch, all the weights are initially randomly initialized. Thus, even if you use the same dataset and hyperparameters, you might end up with a model with a different performance. This aspect can be solved by always using a seed before generating random numbers, as in reality, we cannot digitally create randomness, only pseudo-random numbers. Thus, by providing a seed, we ensure that we always produce the same trace of pseudo-random numbers. This can also happen at the feature engineering step, in case we impute values with random values or randomly remove data or labels. But as a general rule of thumb, always try to make your processes as deterministic as possible, and in case you have to introduce randomness, always provide a seed that you have control over.

**Unlock this book's exclusive benefits
now**

<https://www.packtpub.com/unlock/9781>

This book comes with additional benefits **836200079**
designed to elevate your learning
experience.

*Note: Have your purchase invoice ready
before you begin.*

Answers collapsed